

Observer în academy — rezolvarea

Pattern-ul Observer aplicat pe Repository, pas cu pas, cu dovada prin rulare.

Scrisă pe codul tău de la sha `067a141`, **compilată și rulată**. Fiecare rezultat de mai jos e copiat din consolă, nu presupus.

De ce ce ai acum nu e Observer

Observer înseamnă două lucruri: **subiectul ține o listă de ascultători** și **anunță în momentul în care se întâmplă ceva**, fără să știe cine ascultă și ce face fiecare.

Nu există listă. În `Common/ITextMapper.cs:3` ai scris `ITextMapper<T> : INotificator`, deci fiecare mapper este notificatorul. Un singur ascultător, ales la compilare: el însuși. Ca să adaugi al doilea, trebuie să deschizi `UserTextMapper.cs` și să-l modifice — exact munca pe care Observer o elimină.

Anunțul pleacă în momentul greșit. `Send` e chemat din `FromText` (`UserTextMapper.cs:72`, `BookTextMapper.cs:18`, `CourseTextMapper.cs:18`, `EnrolmentTextMapper.cs:19`), adică atunci când se *parsează un rând din fișier*, din constructorul lui `Repository`. Momentul real — `Repository.Add`, `Repository.cs:37` — nu cheamă notificatorul deloc, face `Console.WriteLine` direct.

Send nu trimite nimic. Întoarce `Name + ": " + message`. E o metodă de formatare; cine o cheamă decide dacă se aude ceva.

Dovada e chiar în codul tău: la `UserTextMapper.cs:72` chemi `Send(...)` și arunci rezultatul, iar în `BookTextMapper.cs:18` îl pui în `Console.WriteLine`. Pe useri notificarea a dispărut, pe cărți a rămas.

Un notificator adevărat nu poate fi făcut mut de cine îl cheamă. Dacă apelantul, ignorând valoarea de retur, oprește notificarea, atunci notificarea n-a existat niciodată — a existat doar un `string`.

De ce a ajuns acolo

Nu e neatenție. Scriind `ITextMapper<T> : INotificator`, ai obligat **fiecare mapper** să aibă `Send`. Când compilatorul îți cere o metodă, cauți de unde s-o chemi — iar un mapper are exact două metode:

`ToText` și `FromText` . Interfața a ales locul notificării, nu tu. Și tot ea a obligat `TextMapperCuJurnal` să care `Name` și `Send` , deși un decorator de jurnalizare n-are nicio treabă cu notificarea.

Rezolvarea, în 5 pași

1. Rupii moștenirea

`Common/ITextMapper.cs` — mapperul redevine traducător, atât:

```
namespace stefan_academy_vanilla_charp.Common
{
    public interface ITextMapper<T>
    {
        string ToText(T item);
        T FromText(string text);
    }
}
```

Cad cele 4 metode `Send` din mappere, cele 4 proprietăți `Name` și apelurile din `FromText` . Zgomotul de la pornire dispare ca **efect secundar**, nu ca reparație separată.

2. `Send` devine `void`

`Common/INotificator.cs` :

```
namespace stefan_academy_vanilla_charp.Common
{
    public interface INotificator
    {
        string Name { get; }
        void Send(string message);
    }
}
```

Cât timp întorcea `string` , apelantul decidea dacă se aude ceva. Cu `void` , ascultătorul e obligat să facă el ceva.

3. `Repository<T>` devine subiectul

E singura clasă care știe *când* se întâmplă ceva. În `Common/Repository.cs` adaugi câmpul și cele două metode:

```
private readonly List<INotificator> ascultatori = new();

public void Subscribe(INotificator ascultator)
{
    ascultatori.Add(ascultator);
}

private void Anunta(string message)
{
    foreach (INotificator ascultator in ascultatori)
    {
        ascultator.Send(message);
    }
}
```

`Add` (:37) și `Remove` (:44) cheamă `Anunta(...)` în loc de `Display(...)` , iar `IDisplay` iese din declarația clasei. Consola nu mai e bătută în cuie în `Repository` — devine un ascultător ca oricare altul.

4. Doi ascultători concreți

`Common/NotificatorConsola.cs` :

```
namespace stefan_academy_vanilla_charp.Common
{
    public class NotificatorConsola : INotificator
    {
        public string Name
        {
            get { return nameof(NotificatorConsola); }
        }

        public void Send(string message)
        {
            Console.WriteLine(Name + ": " + message);
        }
    }
}
```

Common/NotificatorFisier.cs :

```

namespace stefan_academy_vanilla_charp.Common
{
    public class NotificatorFisier : INotificator
    {
        private readonly string cale;

        public NotificatorFisier(string cale)
        {
            this.cale = cale;
        }

        public string Name
        {
            get { return nameof(NotificatorFisier); }
        }

        public void Send(string message)
        {
            using var writer = new StreamWriter(cale, true);
            writer.WriteLine(Name + ": " + message);
        }
    }
}

```

5. Legarea

În `ViewLogIn.cs:10` linia se sparge în trei, fiindcă azi repository-ul e construit inline, ca argument:

```

Users.Repositories.UserRepository repository = new();
repository.Subscribe(new NotificatorConsola());
UserService service = new(repository);

```

În `ViewStudent.cs:15-17` cele trei servicii sunt inițializatori de câmp, deci se mută în constructor — altfel n-ai unde să chemi `Subscribe` :

```
private CourseService courseService;
private BookService bookService;
private EnrolmentService enrolmentService;

private User loggedUser;

public ViewStudent(User user)
{
    loggedUser = user;

    BookRepository bookRepository = new();
    bookRepository.Subscribe(new NotificatorConsola());
    bookRepository.Subscribe(new NotificatorFisier(
        Path.Combine("..", "..", "..", "Data", "jurnal.txt")));
    bookService = new(bookRepository);

    courseService = new(new Courses.Repositories.CourseRepository());
    enrolmentService = new(
        new Enrolments.Repositories.EnrolmentRepository());
}
```

Ce se întâmplă cu decoratorul tău

După pasul 1, `TextMapperCuJurnal` nu mai e obligat să aibă `Name` și `Send`. Rămâne doar ce ține de jurnalizare:

```

public string ToText(T item)
{
    Scribe(typeof(T).Name + " s-a transformat in text");
    return interior.ToText(item);
}

public T FromText(string text)
{
    Scribe("textul s-a transformat in " + typeof(T).Name);
    return interior.FromText(text);
}

private void Scribe(string message)
{
    string cale = Path.Combine("..", "..", "..",
                                "Data", "jurnal.txt");
    using var writer = new StreamWriter(cale, true);
    writer.WriteLine(message);
}

```

Jumătate din clasă a dispărut — și e fix jumătatea care n-avea legătură cu jurnalul. **Ăsta e costul lui `ITextMapper<T> : INotificator`, măsurat în linii.**

Cele două  din runda 8 se închid aici, nu separat: `nameof(T)` scria literal „T” fiindcă `nameof` lucrează la **compilare**, pe numele simbolului — tipul real se află abia la rulare, prin `typeof(T).Name`. Iar `new StreamWriter(cale)` deschide fișierul în mod *truncate*, deci îl golea la fiecare apel; `new StreamWriter(cale, true)` adaugă la sfârșit.

Dovada prin rulare

Înainte — pornire, login ca student, codul tău de la `067a141` :

```
=====LOG IN=====
```

```
Numele: Prenumele: CourseTextMapper Course Analizacreated successfully
CourseTextMapper Course Algebracreated successfully
... (încă 27 de linii) ...
EnrolmentTextMapper Enrolment created successfully
Logat cu succes!
```

29 de linii de zgomot între prompt și „Logat cu succes!”: 6 cursuri + 7 cărți + **16** pentru cele 8 înscrieri (fișierul se citește de două ori — restanța B4 din runda 7).

După — aceeași apăsare de taste:

```
=====LOG IN=====
```

```
Numele: Prenumele: Logat cu succes!
Apasati tasta 0 pentru a iesi
```

Zero. Nu pentru că am șters un `WriteLine`, ci pentru că `Read()` rulează în constructor, iar `Subscribe` vine **după** — la încărcare nu mai are cine să audă.

Testul care contează — un singur `Add`, doi ascultători. Adaugi o carte din meniu:

```
Tastati numele cartii: NotificatorConsola: Repository: Item was added
Cartea CarteObserver adaugata cu succes la data de 30-09-2026
```

iar în `Data/jurnal.txt`, în aceeași rulare:

```
textul s-a transformat in User
... (8 randuri, unul per user) ...
NotificatorFisier: Repository: Item was added
```

Un eveniment, două reacții, `Repository.cs` neschimbat între cele două. Plus: jurnalul păstrează acum toate cele 8 rânduri și scrie `User`, nu „T”.

De verificat de tine

1. Adaugă un al treilea ascultător **fără să atingi Repository.cs** . Dacă trebuie să-l deschizi, ceva din pașii 3-5 nu e pus unde trebuie.
2. Scoate **NotificatorConsola** de pe **UserRepository** și pornește aplicația: login-ul trebuie să meargă identic. Subiectul nu depinde de ascultători.
3. Răspunde în scris: **ce a dispărut din TextMapperCuJurnal după pasul 1 și de ce?**



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro